

ASSIGNMENT 1 - TASK 2

REPORT ON CIFAR-10 IMAGE CLASSIFICATION USING K-NEAREST NEIGHBORS

Student Name	Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number	2621MA08, 2621MA12, 2621MA01
Group	08
Course	Large Language Models
Submission Date	16/08/2026

1. Introduction

In this project, the K-Nearest Neighbors (KNN) classification algorithm is implemented from scratch and applied to the CIFAR-10 image dataset.

CIFAR-10 contains color images belonging to 10 different classes. Each image has a resolution of 32×32 pixels with three color channels. Since KNN works by calculating the distance between data points, the images are converted into flattened feature vectors before classification.

The main purpose of this project is to understand the working of KNN without directly using a pre-built KNN classifier. Different distance measures and different values of K are tested to determine which combination provides the best classification accuracy.

2. Objectives

1. To load and extract the CIFAR-10 dataset.
2. To understand the structure and dimensions of the dataset.
3. To implement the KNN algorithm from scratch.
4. To implement different distance metrics.
5. To test KNN on a smaller subset of CIFAR-10.
6. To compare the performance of different K values.
7. To compare Euclidean, Manhattan, Cosine and Hamming distance metrics.
8. To identify the best-performing combination of K and distance metric.
9. To evaluate the best model using a confusion matrix and classification report.
10. To visualize the effect of K values and distance metrics on classification accuracy.

3. Dataset Description

The dataset used in this project is CIFAR-10.

The dataset consists of:

- 50,000 training images
- 10,000 test images
- 10 different image classes
- Image size: 32×32 pixels
- 3 color channels (RGB)
- 3,072 features per image after flattening

The loaded data had the following dimensions:

Training features: (50,000, 3,072)

Training labels: (50,000,)

Test features: (10,000, 3,072)

Test labels: (10,000,)

The images were reshaped to (50,000, 32, 32, 3) and (10,000, 32, 32, 3) for image representation. For KNN calculations, the flattened 3,072-dimensional representation was used.

4. Tools and Libraries Used

The project was implemented in Python using Google Colab.

Main libraries:

- NumPy – numerical calculations and array manipulation
- tarfile – extraction of the compressed CIFAR-10 dataset
- os – file and directory handling
- pickle – loading CIFAR-10 batch files
- collections.Counter – majority voting in KNN
- Matplotlib – visualization
- Seaborn – confusion matrix visualization
- Scikit-learn metrics – confusion matrix and classification report

5. Dataset Extraction and Loading

The CIFAR-10 compressed file was extracted using Python's tarfile module. The compressed dataset used in the notebook was /content/cifar-10-python.tar.gz.

The extracted CIFAR-10 batch files were loaded using pickle. The load_cifar_batch() function reads individual batches and retrieves image data and labels. The load_cifar10_data() function loads the five training batches, combines them using NumPy concatenation, and loads the test batch.

6. Data Preprocessing

Each CIFAR-10 image contains:

$32 \times 32 \times 3 = 3,072$ pixel values.

The loaded training and test data initially had shapes (50,000, 3,072) and (10,000, 3,072). For visualization, the images were reshaped into $32 \times 32 \times 3$ format. Before applying KNN, the data was converted back to flattened feature vectors because the implemented distance functions operate on vectors.

7. KNN Algorithm Implementation

The K-Nearest Neighbors classifier was implemented from scratch. KNN stores the training data rather than constructing a conventional mathematical model.

For every test image, the implementation:

1. Calculates the distance between the test image and every training image.
2. Sorts the calculated distances.
3. Selects the K nearest samples.
4. Obtains their class labels.

5. Performs majority voting.
6. Assigns the most common class as the prediction.

The KNNClassifier class contains training and prediction methods. The train() method stores the training data, while _predict_single() calculates distances and performs majority voting.

8. Distance Metrics

Four distance metrics were used in the final experiments.

8.1 Euclidean Distance

$$d(x,y) = \sqrt{\sum (x_i - y_i)^2}$$

8.2 Manhattan Distance

$$d(x,y) = \sum |x_i - y_i|$$

8.3 Cosine Distance

Cosine distance was calculated as $1 - \text{cosine similarity}$.

8.4 Hamming Distance

Hamming distance counts the number of positions at which two vectors contain different values.

Minkowski distance was also implemented in the notebook, using $p = 3$, although it was not included in the final comparison experiment.

9. Initial KNN Testing

Because KNN on high-dimensional CIFAR-10 data is computationally expensive, the implementation was first tested on a subset of 5,000 training samples and 1,000 test samples.

For $K = 3$, the obtained accuracies were:

Euclidean: 19.10%

Manhattan: 23.80%

Cosine: 10.30%

Hamming: 23.20%

Manhattan distance performed best in this initial test.

10. Experimentation with Different K Values

The following K values were tested:

$K = 1, 3, 5, 7$

The four distance metrics were Euclidean, Manhattan, Cosine and Hamming. Therefore, 16 combinations of K values and distance metrics were evaluated. The experiments used 5,000 training samples and 1,000 test samples.

11. Experimental Results

The results obtained from the notebook are:

K	Euclidean	Manhattan	Cosine	Hamming
1	17.90%	23.20%	10.90%	22.80%
3	19.10%	23.80%	10.30%	23.20%
5	20.50%	25.40%	11.00%	24.60%
7	20.80%	26.20%	10.80%	25.90%

The best configuration was K = 7 with Manhattan distance, achieving an accuracy of 26.20%.

12. Analysis of Results

The experimental results show that the choice of distance metric has a significant effect on KNN performance.

Euclidean accuracy increased from 17.90% at K = 1 to 20.80% at K = 7.

Manhattan distance provided the best overall performance, increasing from 23.20% at K = 1 to 26.20% at K = 7.

Cosine distance performed poorly for the given image representation, with accuracy remaining around 10–11%.

Hamming distance performed better than Euclidean and Cosine for most tested K values and reached 25.90% at K = 7.

Overall, Manhattan distance with K = 7 was the best tested configuration.

13. Evaluation of the Best Model

The best model was evaluated using a confusion matrix and a classification report containing precision, recall and F1-score.

Selected model:

K = 7

Distance metric = Manhattan

Accuracy = 26.20%

14. Confusion Matrix Analysis

The confusion matrix contains 10 rows and 10 columns corresponding to the ten CIFAR-10 classes.

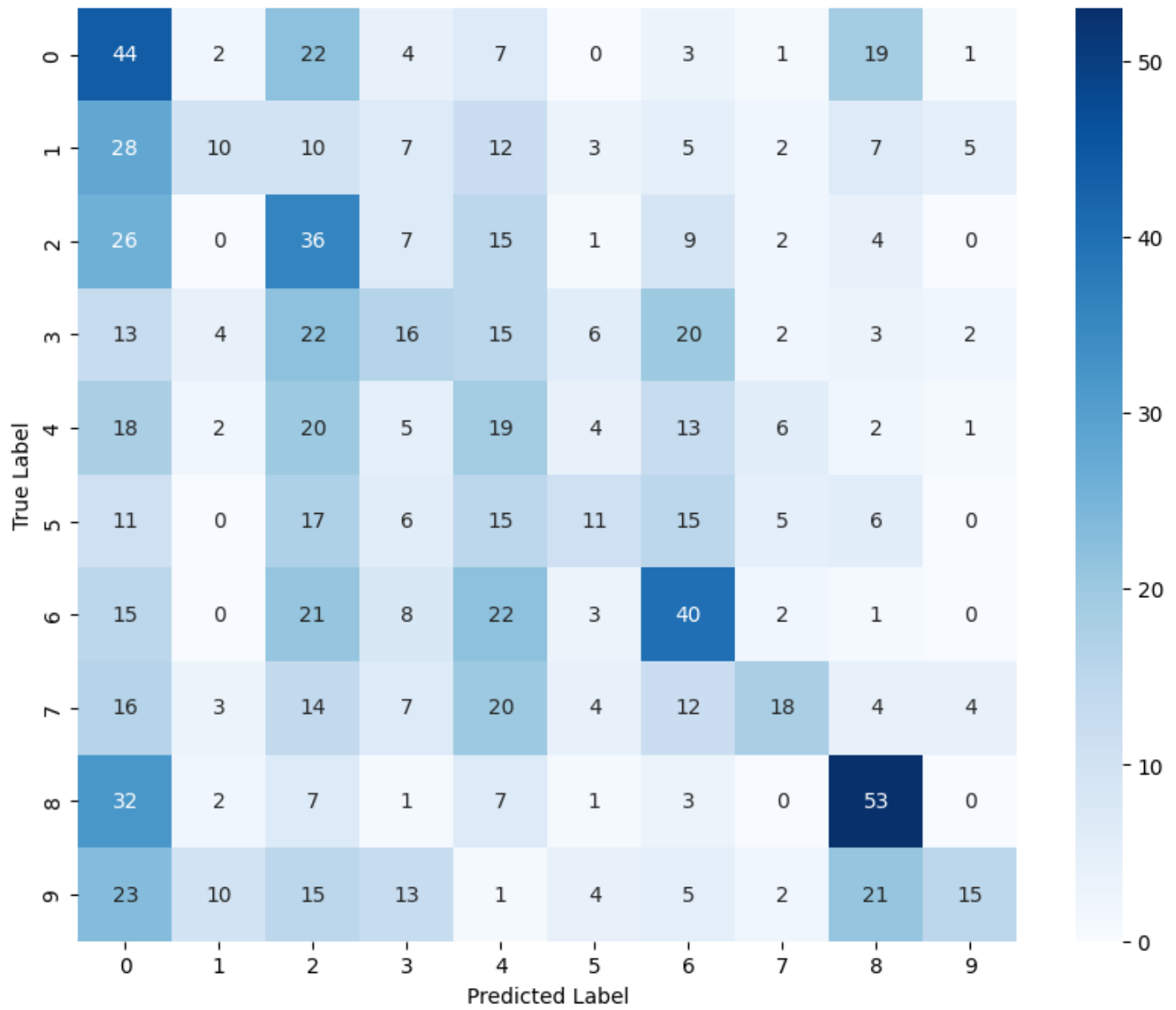
Diagonal values represent correctly classified samples, while off-diagonal values represent misclassified samples.

--- Evaluation for Best Model (K=7, Metric=manhattan) ---

Confusion Matrix:

```
[[44  2 22  4  7  0  3  1 19  1]
 [28 10 10  7 12  3  5  2  7  5]
 [26  0 36  7 15  1  9  2  4  0]
 [13  4 22 16 15  6 20  2  3  2]
 [18  2 20  5 19  4 13  6  2  1]
 [11  0 17  6 15 11 15  5  6  0]
 [15  0 21  8 22  3 40  2  1  0]
 [16  3 14  7 20  4 12 18  4  4]
 [32  2  7  1  7  1  3  0 53  0]
 [23 10 15 13  1  4  5  2 21 15]]
```

Confusion Matrix for Best KNN Model (K=7, Metric=manhattan)



Classification Report:

	precision	recall	f1-score	support
0	0.19	0.43	0.27	103
1	0.30	0.11	0.16	89
2	0.20	0.36	0.25	100
3	0.22	0.16	0.18	103
4	0.14	0.21	0.17	90
5	0.30	0.13	0.18	86
6	0.32	0.36	0.34	112
7	0.45	0.18	0.25	102
8	0.44	0.50	0.47	106
9	0.54	0.14	0.22	109
accuracy			0.26	1000
macro avg	0.31	0.26	0.25	1000
weighted avg	0.31	0.26	0.25	1000

The confusion matrix shows that the KNN classifier has difficulty separating several visually similar CIFAR-10 categories.

15. Classification Report

The classification report obtained from the notebook was:

Class 0: Precision 0.19, Recall 0.43, F1-score 0.27

Class 1: Precision 0.30, Recall 0.11, F1-score 0.16

Class 2: Precision 0.20, Recall 0.36, F1-score 0.25

Class 3: Precision 0.22, Recall 0.16, F1-score 0.18

Class 4: Precision 0.14, Recall 0.21, F1-score 0.17

Class 5: Precision 0.30, Recall 0.13, F1-score 0.18

Class 6: Precision 0.32, Recall 0.36, F1-score 0.34

Class 7: Precision 0.45, Recall 0.18, F1-score 0.25

Class 8: Precision 0.44, Recall 0.50, F1-score 0.47

Class 9: Precision 0.54, Recall 0.14, F1-score 0.22

Overall accuracy: 0.26 (26%)

Macro-average precision: 0.31

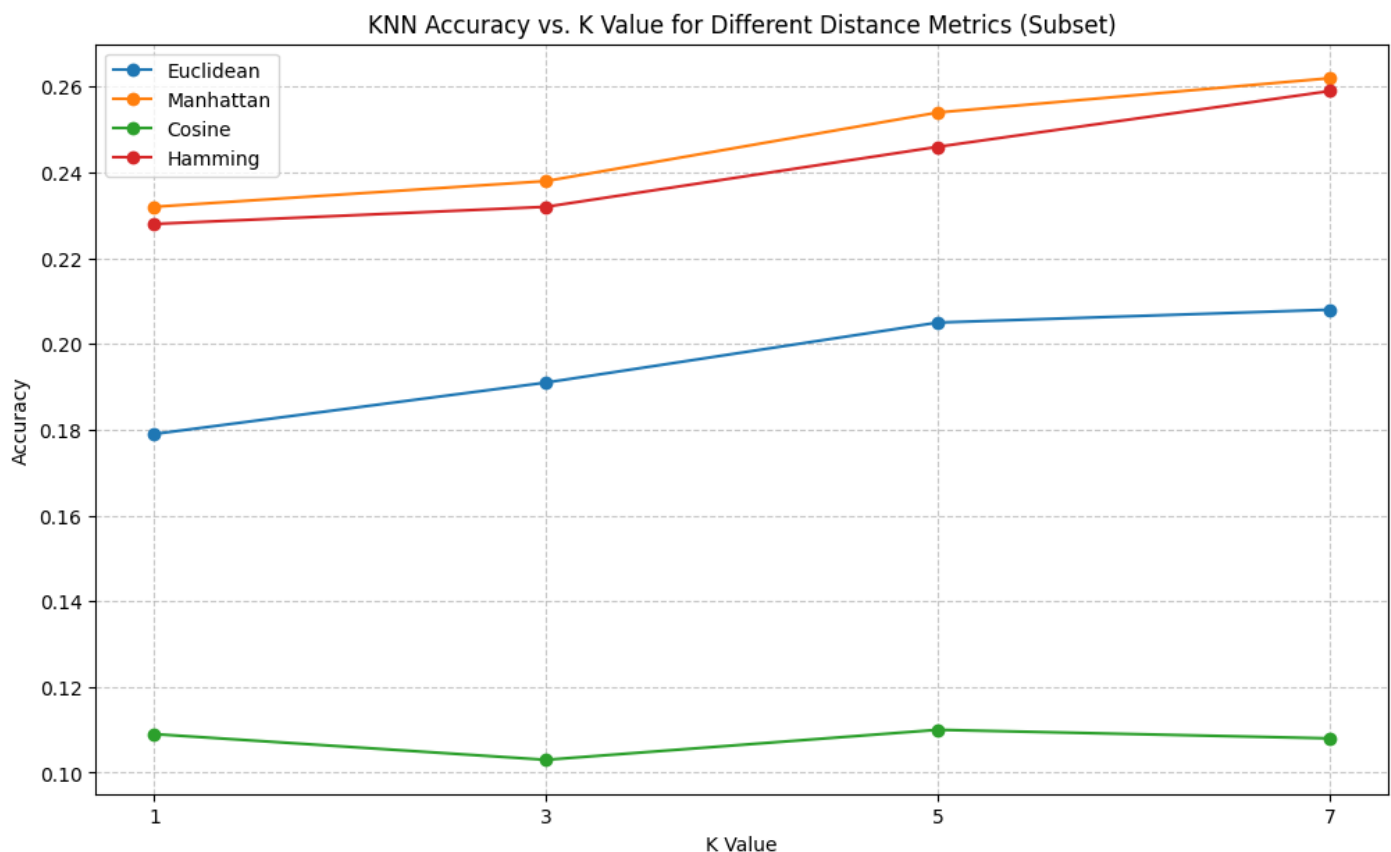
Macro-average recall: 0.26

Macro-average F1-score: 0.25

Class 8 achieved the highest F1-score of 0.47, while class 9 achieved the highest precision of 0.54.

16. Visualization

The notebook visualizes the effect of different K values on classification accuracy. K value is placed on the X-axis and accuracy on the Y-axis, with separate lines representing the different distance metrics. This makes it easier to compare model performance as K changes.



17. Computational Considerations

KNN is computationally expensive because prediction requires distance calculations between test samples and training samples. CIFAR-10 contains 50,000 training images, each represented by 3,072 features.

For this reason, the notebook uses a subset of 5,000 training samples and 1,000 test samples for the experiments. This makes experimentation more practical while still demonstrating the working and performance of the KNN implementation.

18. Advantages

1. KNN was implemented from scratch.
2. Multiple distance metrics were implemented and compared.
3. Different K values were experimentally evaluated.
4. The model was evaluated using accuracy, confusion matrix, precision, recall and F1-score.
5. Visualization was used to analyze the relationship between K and accuracy.

19. Limitations

1. The best accuracy was relatively low at 26.20%.
2. KNN requires many distance calculations during prediction.
3. CIFAR-10 images have a high-dimensional representation.
4. Raw pixels are not necessarily the most effective image representation.
5. The experiments were performed on a subset of the complete dataset.
6. The from-scratch implementation is slower than optimized nearest-neighbor implementations.
7. Hamming distance is not necessarily an ideal measure of visual similarity for image data.

20. Conclusion

This project successfully implemented a K-Nearest Neighbors classifier from scratch for CIFAR-10 image classification.

The CIFAR-10 dataset was extracted and loaded successfully, producing 50,000 training samples and 10,000 test samples. Images were represented as 3,072-dimensional feature vectors for distance calculations.

Four distance metrics—Euclidean, Manhattan, Cosine and Hamming—were compared using K values of 1, 3, 5 and 7.

The best tested configuration was $K = 7$ with Manhattan distance, achieving 26.20% accuracy on the 1,000-image test subset.

Although the accuracy was modest, the project demonstrates the complete process of implementing, testing and evaluating KNN on a high-dimensional image classification problem. It also demonstrates that KNN can become computationally expensive when the dataset contains a large number of high-dimensional samples.